

ESTÁNDAR INTERNO DE EQUIPO

Guía para mantener código escalable

Criterios prácticos para diseñar, documentar, probar, revisar y automatizar el software del proyecto.

Objetivo

Que cada cambio deje el sistema igual o mejor: más claro, probado y fácil de modificar.

Regla central

La calidad no se mide por pocas líneas, sino por responsabilidades claras y cambios seguros.

Incluye

Diseño y estructura / Documentación / Pruebas / Git y revisión / Integración continua / Seguridad

Versión 1.0 - Agosto de 2026

1. Principios que guían nuestras decisiones

Estas reglas sirven para decidir cuándo hay dudas. Son más útiles que una lista de prohibiciones sin contexto.

Principio	Cómo se aplica
Claridad antes que ingenio	Otro integrante debe comprender el cambio sin que su autor tenga que explicarlo.
Una responsabilidad por unidad	Cada clase y método debe tener una razón clara para cambiar.
Cambios pequeños y verificables	Un commit o PR debe resolver una idea concreta y poder revisarse con seguridad.
Pruebas junto al cambio	La lógica nueva o corregida se acompaña de pruebas proporcionales a su riesgo.
Automatizar lo repetible	Compilación, pruebas y revisiones mecánicas se ejecutan por herramientas, no por memoria.

Sobre el límite de 60 líneas. Es una alerta útil, no una meta. Si un método supera aproximadamente 40-60 líneas, se revisa si mezcla etapas, tiene demasiadas decisiones o puede extraerse una operación con nombre propio. No se crean métodos pequeños que solo escondan el flujo real.

Se considera una mala señal cuando una función lee datos, valida, calcula, persiste y actualiza la interfaz. También cuando requiere muchos comentarios para separar bloques internos. La solución normal es extraer métodos por significado, no cortar por cantidad de líneas.

2. Estándar de código

El objetivo es que el código exprese el dominio del proyecto y que las reglas importantes no puedan romperse desde cualquier archivo.

Nombres. Clases con sustantivos: CuentaBancaria. Métodos con verbos: registrarDeposito(). Variables que expliquen la información: saldoDisponible, no x o dato.

Encapsulamiento. Los atributos deben ser privados. Una clase controla sus invariantes: por ejemplo, solo CuentaBancaria decide si un retiro es válido; la interfaz no modifica el saldo directamente.

```
public boolean retirar(BigDecimal monto) {
    if (monto.signum() <= 0 || monto.compareTo(saldo) > 0) {
        return false;
    }
    saldo = saldo.subtract(monto);
    return true;
}
```

Metodo	Acuerdo del equipo
Tamaño	Preferiblemente corto y cohesivo. Revisar a partir de 40-60 líneas; no hay límite obligatorio.
Parámetros	Evitar listas largas. Si varios datos viajan juntos, crear un objeto que represente ese concepto.
Duplicación	Extraer lógica repetida solo cuando existe una regla común real y un nombre claro para ella.
Errores	No silenciar excepciones. Traducir errores a mensajes útiles y conservar detalle técnico en logs seguros.
Recursos	Cerrar archivos y conexiones con try-with-resources. Para dinero, evitar double: usar BigDecimal o centavos enteros.

3. Estructura, documentación y decisiones

La escalabilidad aparece cuando el proyecto puede crecer sin que cada funcionalidad obligue a tocar todos los archivos.

Separar por responsabilidad reduce acoplamiento. Para un proyecto Java, una estructura inicial razonable puede ser:

```
src/main/java/com/equipo/proyecto/  
model/ # Entidades y reglas propias del dominio  
service/ # Casos de uso y lógica de aplicación  
repository/ # Acceso a SQLite u otra persistencia  
ui/ # Menús, vistas o controladores  
validation/ # Validaciones reutilizables  
src/test/java/ # Pruebas con la misma estructura de paquetes  
docs/ # Arquitectura, decisiones, API y guía de ejecución
```

La estructura no debe ser dogmática. En proyectos pequeños puede haber menos capas; la regla es que la interfaz, la lógica de negocio y la base de datos no queden mezcladas en un mismo método o clase.

Documentación mínima obligatoria

- README: propósito, requisitos, cómo ejecutar, cómo probar, estructura de carpetas y miembros del equipo.
- Decisiones técnicas: una nota corta cuando elijan una base de datos, patrón, librería o regla importante. Debe explicar el por qué, no solo el qué.
- API pública y reglas de negocio: documentar métodos no evidentes, precondiciones, efectos y errores esperados.
- Comentarios: explicar una decisión, restricción o consecuencia. No repetir lo que ya expresa el código.

Ejemplo de comentario útil: `// Se rechaza el retiro para impedir sobregiros.` Ejemplo inútil: `saldo -= monto; // resta monto al saldo.`

Regla de diseño: antes de crear una clase o patrón, el equipo debe poder responder: qué problema actual resuelve, qué dependencia evita y cómo se probará. No se agrega complejidad por anticipación.

4. Pruebas que dan confianza

Una prueba no es solo para encontrar bugs: también protege el comportamiento cuando alguien refactoriza el código más adelante.

La mayor parte de las pruebas deben ser unitarias: rápidas, deterministas y centradas en una regla. Agregar pruebas de integración donde se conecte la base de datos, archivos o servicios. Reservar las pruebas de interfaz o extremo a extremo para flujos críticos.

Nivel	Que verifica	Ejemplo en MiniBank
Unitaria	Una regla o clase aislada.	No permite retirar un monto mayor al saldo.
Integración	Colaboración entre componentes reales.	Repositorio guarda y recupera un cliente en SQLite temporal.
Flujo crítico	Recorrido visible para el usuario.	Crear cuenta, depositar y consultar saldo final.

Casos que siempre se prueban

- Camino normal: la operación funciona con datos válidos.
- Límites: cero, vacío, valor mínimo, valor máximo y saldo exacto.
- Entradas inválidas: texto en un campo numérico, correo inválido, datos duplicados o retiro insuficiente.
- Regresión: si se corrige un bug, agregar una prueba que fallaba antes de corregirlo.

Eviten medir calidad solo con porcentaje de cobertura. Una prueba útil verifica comportamiento relevante; una prueba que solo ejecuta líneas sin comprobar resultados puede dar falsa confianza.

5. Git, revisión y definición de terminado

El historial y la revisión son parte del producto. Un cambio que no puede revisarse con facilidad tampoco puede mantenerse con facilidad.

Convención de commits. Un commit representa una idea completa y relacionada. Usar verbo en presente y especificar el resultado:

Evitar	Preferir
cambios finales	Agrega validación de retiros sin saldo
arreglo	Corrige persistencia duplicada de clientes
update	Añade pruebas para depósito con monto cero

Checklist de revisión por Pull Request

- El PR resuelve un solo objetivo y explica qué cambio, por qué y cómo fue probado.
- No mezcla refactorización grande, funcionalidad nueva y cambios visuales sin relación.
- Los nombres, responsabilidades y errores son claros; no hay código muerto ni duplicado injustificado.
- Las pruebas nuevas pasan y cubren el comportamiento modificado, incluidos casos límite relevantes.
- No incluye secretos, contraseñas, llaves API, archivos de compilación ni bases de datos locales.
- Al menos otro integrante revisa los cambios importantes antes de fusionarlos.

Definition of Done (DoD). Una tarea no está terminada cuando 'funciona en mi computadora'. Está terminada cuando cumple el objetivo, está documentada, tiene pruebas pertinentes, pasa el pipeline y puede revisarse sin ambigüedad.

6. Integración continua con GitHub Actions

Aprender GitHub Actions es una muy buena siguiente etapa: la pipeline ejecuta las verificaciones básicas automáticamente en cada push y pull request.

Para un proyecto Java con Maven, una primera pipeline debe: obtener el código, configurar Java, restaurar dependencias y ejecutar `mvn test`. Cuando eso sea estable, pueden sumar `formatter`, análisis estático y pruebas de integración.

Archivo sugerido: `.github/workflows/ci.yml`

```
name: CI Java

on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - name: Ejecutar pruebas
        run: mvn --batch-mode test
```

Antes de activar el archivo, ajusten `java-version` a la versión real del proyecto. Si todavía no usan Maven, la prioridad es migrar a Maven o Gradle antes de crear una pipeline compleja: un proyecto debe tener un comando reproducible para compilar y probar.

Etapas recomendadas

Etapas	Ahora	Después
Build	Compilar el proyecto.	Verificar que el empaquetado genere un artefacto.
Tests	Ejecutar JUnit en cada PR.	Agregar integración con SQLite temporal y reportes.
Calidad	Formatear y revisar compilación sin warnings graves.	Analizadores estáticos como Checkstyle o SpotBugs.
Seguridad	No subir secretos; usar variables protegidas si algún día se requieren.	Escanear dependencias y actualizar vulnerabilidades conocidas.

Seguridad básica: validar toda entrada externa, usar `PreparedStatement` para SQL, no exponer detalles internos en mensajes de error y nunca guardar contraseñas o tokens en el repositorio. Las credenciales se excluyen con `.gitignore` y, si hay despliegue, se configuran como secretos del repositorio.

7. Acuerdo operativo del equipo

Estas son las acciones que se aplican desde el siguiente cambio. Son deliberadamente simples para que se conviertan en hábito.

Antes de programar	Durante	Antes de fusionar
Definir el caso de uso, reglas, datos de entrada y resultado esperado.	Crear código por responsabilidad, con nombres claros y validaciones en los límites.	Ejecutar pruebas, revisar diff, actualizar README si cambia cómo usar el proyecto.
Identificar casos límite y decidir qué capa posee cada regla.	Hacer commits pequeños y ejecutar pruebas locales al completar una parte lógica.	Abrir PR enfocado, explicar pruebas y solicitar revisión de un integrante.
Evitar crear abstracciones o librerías sin una necesidad actual.	Agregar o actualizar prueba cuando se agrega una regla o se corrige un bug.	Confirmar que CI pasa y que no se incluyeron secretos ni archivos generados.

Conclusión. Un proyecto mantenible no depende de que una persona recuerde todo. Depende de que el equipo convierta sus decisiones en estructura, pruebas, documentación, revisiones y automatizaciones repetibles. Esta guía debe actualizarse cuando el proyecto encuentre una regla que necesite formalizar.

Fuentes consultadas

- Google. Google Java Style Guide. <https://google.github.io/styleguide/javaguide.html>
- Google Engineering Practices. What to look for in a code review. <https://google.github.io/eng-practices/review/reviewer/looking-for.html>
- Google Engineering Practices. Small CLs. <https://google.github.io/eng-practices/review/developer/small-cls.html>
- Fowler, M. Code Smell. <https://martinfowler.com/bliki/CodeSmell.html>
- Software Engineering at Google. Why Do We Write Tests? <https://abseil.io/resources/swe-book/html/ch11.html>
- GitHub Docs. Helping others review your changes. <https://docs.github.com/en/pull-requests/concepts/helping-others-review-your-changes>
- GitHub Docs. Building and testing your code. <https://docs.github.com/en/actions/tutorials/build-and-test-code>
- OWASP. Input Validation Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
- Oracle. The try-with-resources Statement. <https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html>